DOCUMENTATION UTILISATEUR

Ce programme offre une petite « arène » graphique pour lancer et visualiser plusieurs simulations (automates et particules) : Conway, Balls, Immigration, Schelling et Boids. Le point d'entrée ouvre une interface Swing qui utilise en Java des fonctionnalités JavaScript, avec des listeners pour gérer les interactions utilisateur afin de rendre l'experience plus agréable.

Prérequis

- Java **JDK 17+** installé (javac / java).
- Le dépôt contient dans `lib/` différentes bibliothèques nécessaires :
  - `lib/gui.jar` utilisé par l'interface graphique.
  - `lib/junit-*.jar` pour les tests unitaires (JUnit 5).
  - `lib/apiguardian-api-1.1.2.jar`, pour les dépendances JUnit.

NB : Le projet compile sous JDK 17+ car il utilise l'API `java.util.random` (introduite dans Java 17). Pour rester compatible avec des versions plus anciennes (JDK 7+), on aurait pu utiliser `ThreadLocalRandom`, comme dans les jeux spaciaux, ou `java.util.Random`.

Compilation

Avec Makefile (recommandé)

- Depuis la racine du projet, compiler avec le Makefile (recommandé) :

```
make compile
```

Compilation manuelle (non recommandée) :

Pour compiler manuellement (sans make) :

```
mkdir -p bin
javac -d bin -sourcepath src -classpath lib/gui.jar $(find src -name "*.java")
```

Exécution

Avec make (recommandé) :

Pour lancez l'interface :

```
make
# ou
make run
```

Remarque : `make run` compile si besoin puis lance l'application graphique.

Execution manuelle (non recommandé) :

Après compilation lancez l'interface :

```
java -classpath "bin:lib/gui.jar" app.Main
```

Remarque : `app.Main` ouvre la fenêtre principale (`Arena of Games`) et ne prend pas d'arguments en ligne de commande — la sélection des simulations se fait via l'interface.

Actions principales

Terminal

- `make compile` : compile les sources vers `bin/` en incluant `lib/gui.jar`.
- `make run` : compile si besoin puis lance l'application graphique.
- `java -classpath "bin:lib/gui.jar" app.Main` : lance directement l'interface.
- `make help` : affiche une aide rapide listant les cibles principales du Makefile (compile, run, test, doc, clean).

Interface graphique (boutons et effets)

- À l'ouverture : cliquer sur "Enter to the arena of games" pour accéder au menu.
- Dans le menu, choisir l'un des simulateurs :
  - Conway — lance `systems.ConwaySimulator` (jeu de la vie, paramètres par défaut : 900×600, couverture par défaut exprimée dans l'UI).
  - Balls — lance `systems.BallsSimulator` (particules/bouncing balls).
  - Immigration — lance `systems.ImmigrationSimulator` (modèle d'immigration/affectation de groupes).
  - Schelling — lance `systems.SchellingSimulator` (modèle de ségrégation de Schelling).
  - Boids — lance `systems.BoidsSimulator` (simulation de boids/essaims ; paramètres par défaut définis dans l'UI).
  - Bouton "Quit" : quitte l'application.
- Chaque simulation ouvre une nouvelle fenêtre avec des contrôles (Play, Pause, Reset, Speed) pour gérer l'exécution.
- Pour lancer une autre simulation, il suffit de selectionner une autre simulation dans l'arène de jeu, ce qui remplacera la fenetre en cours par la nouvelle simulation.

Remarques sur les simulations

- Les paramètres (taille fenêtre, couverture, nombre de groupes, etc.) sont définis dans `app.ui.ArenaFrame` et transmis aux constructeurs des simulateurs ; pour modifier un comportement par défaut éditez ce fichier. Ils sont volontairement codés en dur pour permettre à terme une modification plus conviviale via l'interface utilisateur.
- Chaque bouton appelle `.play()` qui dessine la première frame sur l'instance du simulateur — l'exécution et l'affichage sont gérés côté UI/Swing.

Dépendances et vérifications

- Vérifier que la bibliothèque graphique est présente :

```
ls -l lib/gui.jar
```

- Si `lib/gui.jar` manque, l'UI lèvera des erreurs (ClassNotFoundException). Dans ce cas, soit récupérer `gui.jar` auprès de l'auteur, soit exécuter des simulateurs via un `Main` personnalisé sans dépendances UI (nécessite d'ajouter un petit runner dans `src/`).

Tests

Les tests sont conçus pour être automatisés directement sur le repo git, afin de vérifier la non-régression lors de chaque commit.

Cependant, ils peuvent aussi être exécutés localement pour vérifier le bon fonctionnement des entités et systèmes :

- Les tests sont placés dans le dossier `test/` (packages `entities` et `systems`).
- Le `Makefile` fournit des cibles utiles :
  - `make compile-test` : compile les tests dans `out/test` (utilise les jars JUnit listés dans `lib/`).
  - `make test` (ou `make test-all`) : compile puis exécute tous les tests via `junit-platform-console-standalone`.
  - cibles ciblées : `make test-entities`, `make test-conway`, `make test-immigration`, `make test-schelling`, `make test-boids` pour compiler et exécuter des suites spécifiques.
- Exemple d'exécution :

```
make compile-test
make test-conway
# ou
make test
```

- Sans les jars JUnit, les cibles de test échoueront — vous pouvez installer les jars JUnit manuellement dans `lib/` (voir la section Prérequis).

Crédits

Afin de simplifier l'ajout d'un nouveau type de Boids, en plus du polymorphisme, nous avons utilisé un programme qui transforme facilement les SVG en classes Java/GUI (lien vers le repo git : github.com/Rossi1337/svg2java).